SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences, Chennai

ASSESSMENT TOOL 1

Integrated Experiments

Course Name	INTERNET PROGRAMMING
Course Code	CSA43
Course Outcome	CO-2: Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)
Assessment Tool	Integrated Experiments
Weightage	40%
Total Marks	30

SUBMITTED BY

Student Name	K. Ganesh
Registration No.	192411170
Programme	B.E. Computer Science and Engineering
Institution	SIMATS Engineering, Chennai
Date	21 October 2026

Experiments Covered

- Experiment 1: Responsive Layout Design
- Experiment 2: Form Validation using JavaScript
- Experiment 3: CSS Layout Debugging

Code of Conduct

I, K. Ganesh / 192411170 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Experiment 1: Responsive Layout Design

Scenario: a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

(a) Analysis of reasons for lack of responsiveness

- **Missing viewport meta tag:** mobile browsers render at ~980px and shrink the page, so nothing adapts.
- **Fixed pixel widths** (e.g. width:1200px) and fixed heights force horizontal scrolling and clipped content.
- **Float / absolute-position layouts** that do not reflow, plus missing clearfix causing collapsed containers.
- **No media queries**, so the desktop layout is applied on every screen size.
- **Non-fluid media:** images and videos without max-width:100% overflow their containers.
- **Inconsistent spacing:** hard-coded margins/padding in px, and box-sizing left as content-box so padding adds to widths.

(b) Responsive layout using CSS Grid and Flexbox

A mobile-first design is used. The page is a CSS Grid with named areas (header, nav, main, footer); the navigation and product cards use Flexbox/Grid for distribution. Save as **exp1.html** and open in a browser.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>ShopEasy - Responsive Retail Layout</title>
<style>
  :root { --gap: 1rem; --brand: #1f3a5f; --bg: #f5f7fa; }   *, *::before, *::after { box-sizing:
border-box; margin: 0; padding: 0; }   body { font-family: Arial, sans-serif; background: var(--
bg); color: #222; line-height: 1.5; }   img { max-width: 100%; height: auto; display: block; }

  /* Mobile-first: single column */   .page { display: grid; min-height: 100vh; gap: var(--gap);
padding: var(--gap);           grid-template-areas: "header" "nav" "main" "footer";           grid-
template-rows: auto auto 1fr auto; }   header { grid-area: header; background: var(--brand); color:
#fff; padding: 1rem;            border-radius: 8px; text-align: center; }   nav    { grid-area: nav; }
nav ul { list-style: none; display: flex; flex-wrap: wrap; gap: .5rem; justify-content: center; }   nav
a  { display: block; padding: .6rem 1rem; background: #fff; border-radius: 6px;           text-
decoration: none; color: var(--brand); font-weight: bold;           box-shadow: 0 1px 3px
rgba(0,0,0,.15); }   nav a:hover { background: var(--brand); color: #fff; }   main   { grid-area: main;
display: grid; gap: var(--gap); grid-template-columns: 1fr; }   .card  { background: #fff; padding:
1rem; border-radius: 8px; box-shadow: 0 1px 4px rgba(0,0,0,.12);           display: flex; flex-
direction: column; justify-content: space-between; gap: .5rem; }   .card .price { color: #c0392b; font-
weight: bold; }   .card button { align-self: flex-start; padding: .5rem 1rem; border: 0; border-radius:
6px;                 background: var(--brand); color: #fff; cursor: pointer; }   footer { grid-area:
footer; background: #222; color: #ddd; text-align: center;           padding: 1rem; border-radius:
8px; }

  /* Tablet: 2 product columns, horizontal nav */
@media (min-width: 600px) {
    main { grid-template-columns: repeat(2, 1fr); }
header { text-align: left; }     nav ul { justify-
content: flex-start; }   }
  /* Desktop: sidebar nav + 3 product columns */
  @media (min-width: 992px) {     .page { max-width: 1200px; margin: 0 auto;
grid-template-columns: 220px 1fr;            grid-template-areas: "header
header" "nav main" "footer footer"; }     nav ul { flex-direction: column; }
main { grid-template-columns: repeat(3, 1fr); }   }
</style>
</head>
<body>
<div class="page">
  <header><h1>ShopEasy</h1><p>Everything you need, delivered.</p></header>
  <nav><ul>
    <li><a href="#">Home</a></li><li><a href="#">Men</a></li><li><a href="#">Women</a></li>
    <li><a href="#">Electronics</a></li><li><a href="#">Offers</a></li>
  </ul></nav>
  <main>
```

```
    <article class="card"><h3>Wireless Headphones</h3><p>Noise-cancelling, 30h battery.</p>
      <span class="price">Rs. 2,499</span><button>Add to Cart</button></article>
    <article class="card"><h3>Smart Watch</h3><p>Heart-rate and sleep tracking.</p>
      <span class="price">Rs. 3,999</span><button>Add to Cart</button></article>
    <article class="card"><h3>Backpack</h3><p>Water-resistant, 25L laptop bag.</p>
      <span class="price">Rs. 1,299</span><button>Add to Cart</button></article>
    <article class="card"><h3>Running Shoes</h3><p>Lightweight, breathable mesh.</p>
      <span class="price">Rs. 2,199</span><button>Add to Cart</button></article>
    <article class="card"><h3>Bluetooth Speaker</h3><p>Portable, 12h playtime.</p>
      <span class="price">Rs. 1,799</span><button>Add to Cart</button></article>
    <article class="card"><h3>Sunglasses</h3><p>UV400 polarised lenses.</p>
<span class="price">Rs. 899</span><button>Add to Cart</button></article>
  </main>
  <footer>&copy; 2026 ShopEasy. All rights reserved.</footer>
</div>
</body>
</html>
```

(c) Adaptation across screen sizes (media queries)

Device	Breakpoint	Header / Nav	Content (products)	Layout
Mobile	< 600px (default)	Centred header; nav wraps in a row	1 column	Single-column stacked grid
Tablet	600px - 991px	Left-aligned header; horizontal nav	2 columns	Stacked areas, 2-col card grid
Desktop	≥ 992px	Full-width header; vertical sidebar nav (220px)	3 columns	2-col page grid, max-width 1200px

(d) Spacing, alignment and distribution techniques

- **box-sizing: border-box** and a CSS reset so padding never inflates widths.
- **gap** (grid/flex) with a single **--gap** variable gives uniform spacing instead of per-element margins.
- **justify-content / align-self** in flex cards pin the price and button consistently; **flex-wrap** lets nav links reflow.
- **repeat(n, 1fr)** distributes equal column widths; **max-width + margin:0 auto** centres content on wide monitors.
- Relative units (rem, %, fr) and CSS variables keep spacing, colours and radii consistent across components.

(e) Evaluation and further improvements

Effectiveness: the layout reflows with no horizontal scrolling at 320px, 768px and 1366px; header, navigation, content and footer stay aligned through every breakpoint; images scale within containers; and the single-source CSS is easy to maintain. **Improvements:** use *repeat(auto-fit, minmax(220px, 1fr))* for breakpoint-free product grids; use *clamp()* for fluid typography; add srcset/*picture* for responsive images and lazy loading; add a hamburger menu for mobile; improve accessibility (skip links, focus styles, ARIA landmarks, *prefers-reduced-motion*); and test with Chrome DevTools device mode and Lighthouse.

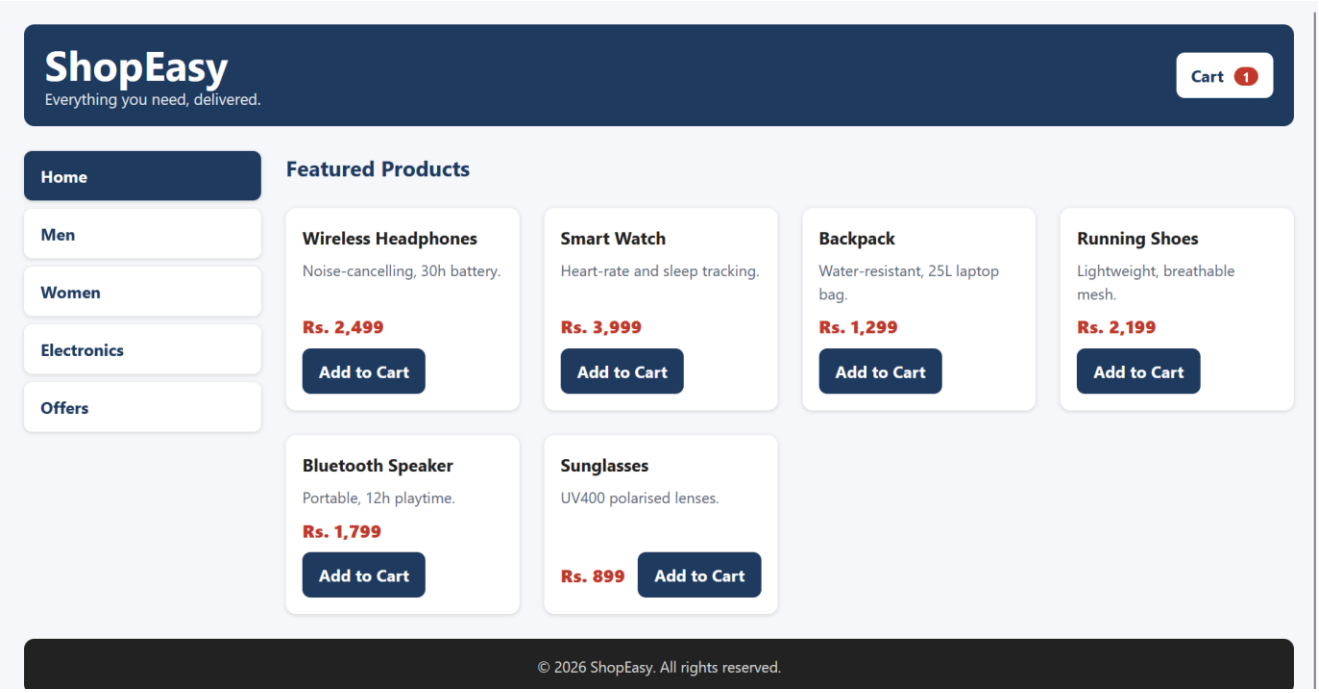


Fig 1: out put

Experiment 2: Form Validation using JavaScript

Scenario: a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

(a) Analysis of issues in the existing validation

- No **type="email"/type="tel"**, **required** or **pattern** attributes, so the browser performs no checks.
- No JavaScript validation, or only a weak check (e.g. just looking for '@'), so values like *a@b* or *12345* pass.
- Validation only on submit (no real-time feedback), and no clear error messages or visual cues.
- Spaces, letters and wrong lengths are accepted for phone numbers; input is not trimmed.
- Relying only on client-side checks: the server must also re-validate, since client code can be bypassed.

(b) Input fields with appropriate HTML attributes

```
<label for="email">Email</label> <input type="email" id="email"
name="email" placeholder="name@example.com"        required
autocomplete="email" aria-describedby="emailErr"> <small class="error"
id="emailErr" role="alert"></small>

<label for="phone">Phone (10 digits)</label> <input type="tel" id="phone"
name="phone" placeholder="9876543210"        required inputmode="numeric"
maxlength="10" pattern="[6-9][0-9]{9}"        autocomplete="tel" aria-
describedby="phoneErr"> <small class="error" id="phoneErr"
role="alert"></small>
```

(c) and (d) JavaScript validation with Regular Expressions and real-time error messages

Complete runnable page (save as **exp2.html**). Validation runs on every keystroke (*input*), when a field loses focus (*blur*) and on submit.

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Registration Form Validation</title>
<style>   * { box-sizing: border-box; font-family: Arial, sans-
serif; }
  body { background: #f5f7fa; display: flex; justify-content: center; padding: 2rem 1rem; }
form { background: #fff; padding: 1.5rem; width: 100%; max-width: 380px; border-radius: 10px;
box-shadow: 0 2px 8px rgba(0,0,0,.15); }   h2 { margin-bottom: 1rem; color: #1f3a5f; }
  label { display: block; margin-top: .8rem; font-weight: bold; }   input { width: 100%; padding: .6rem;
margin-top: .3rem; border: 2px solid #ccc; border-radius: 6px; }   input.valid { border-color: #2e9e5b;
}   input.invalid { border-color: #d93025; }   .error { color: #d93025; display: block; min-height:
1.2em; font-size: .85rem; }   button { margin-top: 1rem; width: 100%; padding: .7rem; border: 0; border-
radius: 6px;            background: #1f3a5f; color: #fff; font-size: 1rem; cursor: pointer; }
button:disabled { background: #9aa7b8; cursor: not-allowed; }   #ok { color: #2e9e5b; margin-top: .8rem;
font-weight: bold; }
</style>
</head>
<body>
<form id="regForm" novalidate>
  <h2>Register</h2>
  <label for="name">Full Name</label>
  <input type="text" id="name" required>
  <small class="error" id="nameErr"></small>

  <label for="email">Email</label>
  <input type="email" id="email" placeholder="name@example.com" required>
  <small class="error" id="emailErr"></small>

  <label for="phone">Phone (10 digits)</label>
  <input type="tel" id="phone" placeholder="9876543210" maxlength="10" required>
  <small class="error" id="phoneErr"></small>

  <button type="submit" id="submitBtn" disabled>Register</button>
  <p id="ok"></p>
</form>

<script>   const EMAIL_RE = /^[^\s@]+@[^\s@]+\.[A-Za-z]{2,}$/;   const PHONE_RE = /^[6-
9]\d{9}$/;           // Indian mobile: 10 digits, starts with 6-9

  const rules = {     name:  v => v.length >= 3 ? "" : "Name must be at least 3
characters.",     email: v => !v ? "Email is required."               :
EMAIL_RE.test(v) ? "" : "Enter a valid email, e.g. name@example.com.",     phone: v =>
!v ? "Phone number is required."               : /\D/.test(v) ? "Only digits are
allowed."
              : v.length !== 10 ? "Phone number must be exactly 10 digits."
: PHONE_RE.test(v) ? "" : "Number must start with 6, 7, 8 or 9."   };

  const form = document.getElementById("regForm");
const btn  = document.getElementById("submitBtn");

  function check(id) {     const input = document.getElementById(id);     const msg =
rules[id](input.value.trim());     document.getElementById(id + "Err").textContent = msg;
input.classList.toggle("invalid", msg !== "");     input.classList.toggle("valid", msg === "" && input.value
!== "");     return msg === "";   }   function checkAll() {     const all = Object.keys(rules).every(id =>
rules[id](document.getElementById(id).value.trim()) === "");     btn.disabled = !all;     return all;   }

  Object.keys(rules).forEach(id => {
const el = document.getElementById(id);
    el.addEventListener("input", () => { check(id); checkAll(); });   // real-time
el.addEventListener("blur",  () => check(id));   });

  form.addEventListener("submit", e => {
    e.preventDefault();
    const ok = Object.keys(rules).map(check).every(Boolean);
    document.getElementById("ok").textContent = ok ? "Registration successful!" : "";
if (ok) form.reset(), btn.disabled = true;   });
</script>
</body>
</html>
```

Regular expressions explained

- **Email** `^[^\s@]+@[^\s@]+\.[A-Za-z]{2,}$`: one or more non-space, non-@ characters; a single @; a domain; a dot; and a TLD of 2+ letters.
- **Phone** `^[6-9]\d{9}$`: exactly 10 digits, first digit 6-9 (valid Indian mobile numbers).

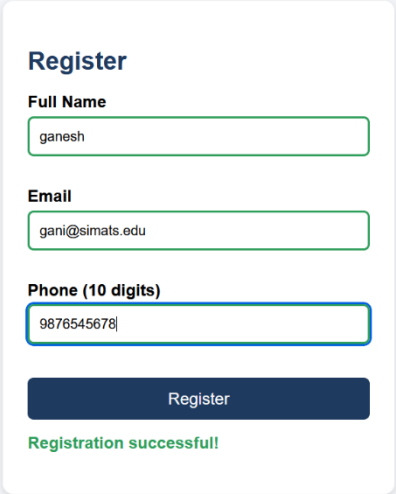
Test results (same patterns executed against sample inputs)

Field	Input	Expected	Result
Email	ganesh@simats.edu	Valid	Valid - Pass
Email	user.name@mail.co.in	Valid	Valid - Pass
Field	**Input**	**Expected**	**Result**
Email	user@domain	Invalid	Invalid - Pass
Email	user@@mail.com	Invalid	Invalid - Pass
Email	user name@mail.com	Invalid	Invalid - Pass
Email	@mail.com	Invalid	Invalid - Pass
Phone	9876543210	Valid	Valid - Pass
Phone	6123456789	Valid	Valid - Pass
Phone	5876543210	Invalid	Invalid - Pass
Phone	98765	Invalid	Invalid - Pass
Phone	98765abcde	Invalid	Invalid - Pass
Phone	98765432101	Invalid	Invalid - Pass

(e) Evaluation and enhancements

Evaluation: all 12 test inputs were classified correctly; users see specific messages as they type, invalid fields turn red, valid ones green, and the Register button stays disabled until every field is valid.
Enhancements:

- **Security:** always re-validate and sanitise on the server (client checks can be bypassed); escape output to prevent XSS; use parameterised queries; add CSRF tokens and rate limiting/CAPTCHA.
- **UX:** show errors only after a field is touched; add country-code selector and input masking (e.g. libphonenumber); suggest typo fixes (gmial.com to gmail.com); add aria-live announcements for screen readers.
- **Verification:** confirm email ownership via a one-time link and phone via an OTP, since a regex proves only the format, not that the address or number exists.
- Debounce validation for slow devices and extend rules to password strength, confirm-password and terms checkbox.



Experiment 3: CSS Layout Debugging

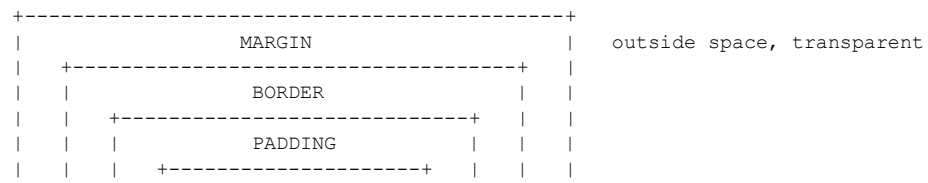
Scenario: a webpage layout overlaps when the browser window is resized, affecting usability.

(a) CSS properties responsible for overlapping

Property / practice	Why it causes overlap
position:absolute with top/left/right in px	Element leaves normal flow; siblings do not reserve its space, so they collide when the viewport shrinks.
Fixed width / height (px)	Content cannot grow or shrink; text overflows into neighbouring blocks.
float without clearfix	Parent collapses and following content wraps under floated items.
Negative margins	Pull elements on top of one another.
box-sizing: content-box	Padding + border are added to the width, so two 50% boxes exceed 100% and wrap or overlap.
overflow: visible (default) / z-index misuse	Overflowing content paints over neighbours; stacking order becomes unpredictable.
white-space: nowrap, no min-width:0 on flex items	Long content refuses to shrink and spills out of its container.

(b) Role of the CSS box model

Every element is a rectangular box made of four layers, from inside to outside: **content** (text/image), **padding** (space inside the border), **border**, and **margin** (space outside, separating neighbours).



```
|   |   |   |       CONTENT       |   |   |   |
|   |   |   +---------------------+   |   |   |
|   |   +-----------------------------+   |   |
|   +-------------------------------------+   |
+-----------------------------------------------+
Total width = content + padding-left/right + border-left/right (+ margin-left/right)
```

- With **content-box**, *width:300px; padding:20px; border:5px* renders 350px wide; with **border-box** it stays 300px, which makes fluid layouts predictable.
- Vertical margins of adjacent blocks *collapse* to the larger value; flex and grid items do not collapse.
- Margin separates elements, padding creates breathing room inside them: mixing these up is a common spacing bug.

(c) Debugging with positioning techniques

Technique	Use in the fix
static / relative	Keep elements in normal flow; relative establishes a reference box for children without moving the element.
absolute	Only for small overlays (badges, tooltips) inside a relative parent, never for main layout blocks.
Technique	**Use in the fix**
flex	One-dimensional rows/columns with wrap, gap and flex:1 so items share space and cannot collide.
grid	Two-dimensional page structure with fr units and minmax() for the sidebar/content split.

Debug procedure: open DevTools, inspect the overlapping element, toggle the box-model overlay to find the offending width/position, resize with device toolbar, and replace fixed values with flexible ones.

(d) Buggy CSS and the corrected CSS

Before (buggy):

```
.container { width: 1000px; }                      /* fixed width */
.sidebar   { position: absolute; left: 0; top: 80px; width: 250px; }
.content   { float: left; width: 750px; margin-left: 250px; padding: 20px; }   /* 1020px total */
.footer    { position: absolute; top: 600px; }     /* collides with content when it grows */
```

After (fixed). Complete runnable page, save as exp3.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>CSS Layout Debugging - Fixed</title>
<style>   *, *::before, *::after { box-sizing: border-box; }     /* predictable
widths */   body { margin: 0; font-family: Arial, sans-serif; background: #f5f7fa;
}

  .container { display: grid; max-width: 1100px; margin: 0 auto; min-height: 100vh;
grid-template-columns: minmax(200px, 250px) 1fr;               grid-template-rows:
auto 1fr auto;               grid-template-areas: "header header" "sidebar content"
"footer footer";               gap: 1rem; padding: 1rem; }

  .header  { grid-area: header; background: #1f3a5f; color: #fff; padding: 1rem; border-radius: 8px; }
  .sidebar { grid-area: sidebar; background: #dde6f0; padding: 1rem; border-radius: 8px; }
.content { grid-area: content; background: #fff; padding: 1.25rem; border-radius: 8px;
min-width: 0;                     /* lets grid item shrink */             overflow-wrap:
```

```
anywhere; }          /* long words cannot spill out */   .footer  { grid-area: footer; background:
#222; color: #ddd; padding: 1rem; border-radius: 8px; }

  /* Absolute positioning is kept only for a small badge inside a relative parent */
  .card  { position: relative; padding: 1rem; border: 1px solid #ccc; border-radius: 8px; }
.badge { position: absolute; top: .5rem; right: .5rem; background: #c0392b; color: #fff;
padding: .15rem .5rem; border-radius: 12px; font-size: .75rem; }

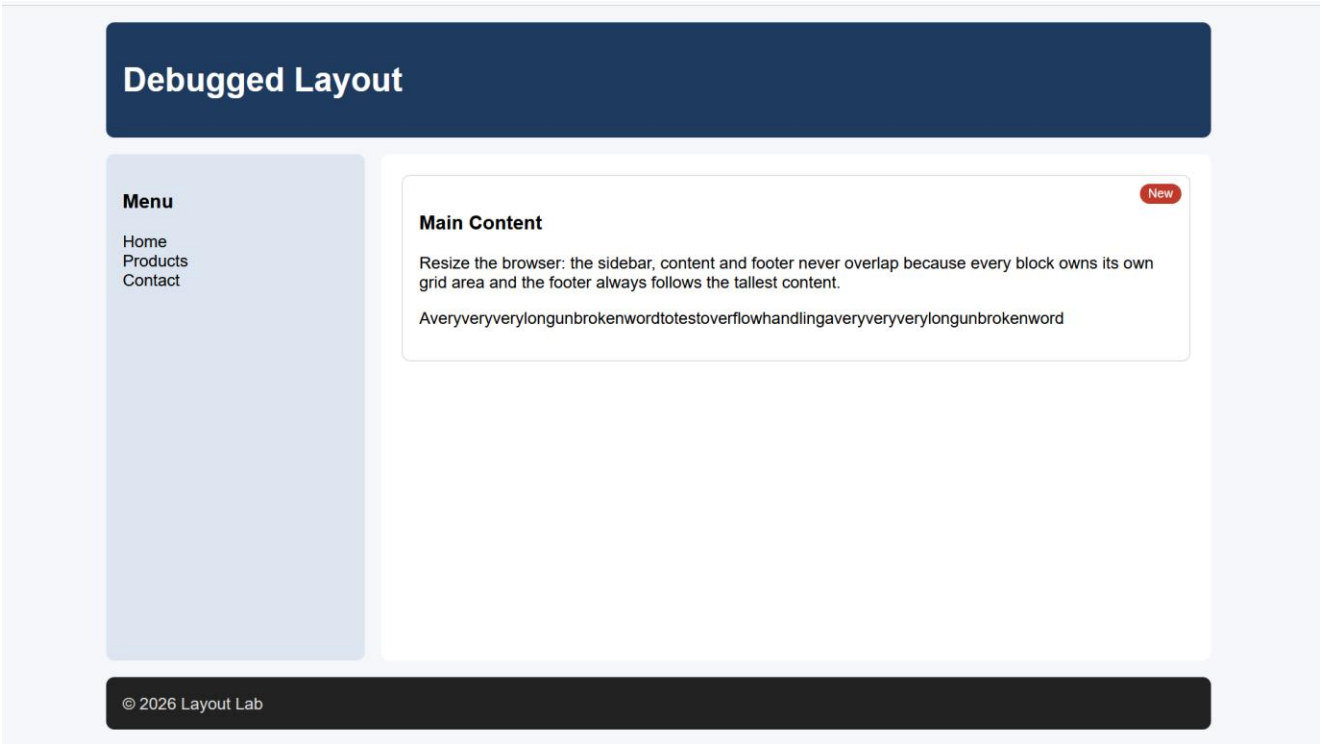
  @media (max-width: 700px) {                    /* stack on small screens */
.container { grid-template-columns: 1fr;                  grid-template-areas:
"header" "sidebar" "content" "footer"; }   }
</style>
</head>
<body>
<div class="container">
  <header class="header"><h1>Debugged Layout</h1></header>   <aside
class="sidebar"><h3>Menu</h3><p>Home<br>Products<br>Contact</p></aside>
  <main class="content">     <div class="card"><span
class="badge">New</span>
      <h3>Main Content</h3>       <p>Resize the browser: the sidebar, content and footer never
overlap because every block          owns its own grid area and the footer always follows the
tallest content.</p>
<p>Averyveryverylongunbrokenwordtotestoverflowhandlingaveryveryverylongunbrokenword</p>
    </div>
  </main>   <footer class="footer">&copy; 2026 Layout
Lab</footer>
</div>
</body>
</html>
```

(e) Evaluation of corrected layout and best practices

Check	Before	After
1200px wide window	Sidebar overlaps content by 20px (1020px in a 1000px box)	Aligned; content fills remaining space
768px wide window	Content overlaps sidebar; footer on top of text	Two columns, no overlap
480px wide window	Horizontal scroll; blocks stacked on each other	Single column stack; no scroll
Long unbroken text	Overflows card	Wraps inside card

Best practices for consistency:

- Apply a global reset with *box-sizing: border-box* and use rem/%/fr/minmax() rather than fixed px widths.
- Use Grid for page structure, Flexbox for components; reserve absolute positioning for small overlays inside a relative parent.
- Design mobile-first, add media queries at content-driven breakpoints, and test at several widths.
- Use CSS variables for spacing and colours; follow a naming convention such as BEM and keep a single stylesheet source of truth.
- Set min-width:0, overflow-wrap and max-width:100% on flexible children; avoid negative margins and magic z-index numbers.
- Validate with DevTools, W3C CSS validator and cross-browser testing; use stylelint for team consistency.



Tools Used and Conclusion

Tool / Technology	Purpose
HTML5	Semantic structure, form input types and validation attributes
CSS3 (Flexbox, Grid, Media Queries, Variables)	Responsive layout, spacing, alignment and debugging fixes
JavaScript (ES6) and Regular Expressions	Real-time client-side form validation
Browser DevTools (Device Mode, Inspector, Console)	Resizing tests, box-model inspection and debugging
VS Code and any modern browser	Code editing and execution

Conclusion. Experiment 1 produced a mobile-first grid layout that adapts across mobile, tablet and desktop. Experiment 2 delivered email and phone validation with regular expressions and live feedback, with server-side checks recommended for security. Experiment 3 identified the CSS causes of overlap, explained the box model and replaced fragile positioning with Grid/Flexbox. Together they apply CO-2 (L3): CSS layout and styling for responsive interfaces plus JavaScript for interactive client-side behaviour.